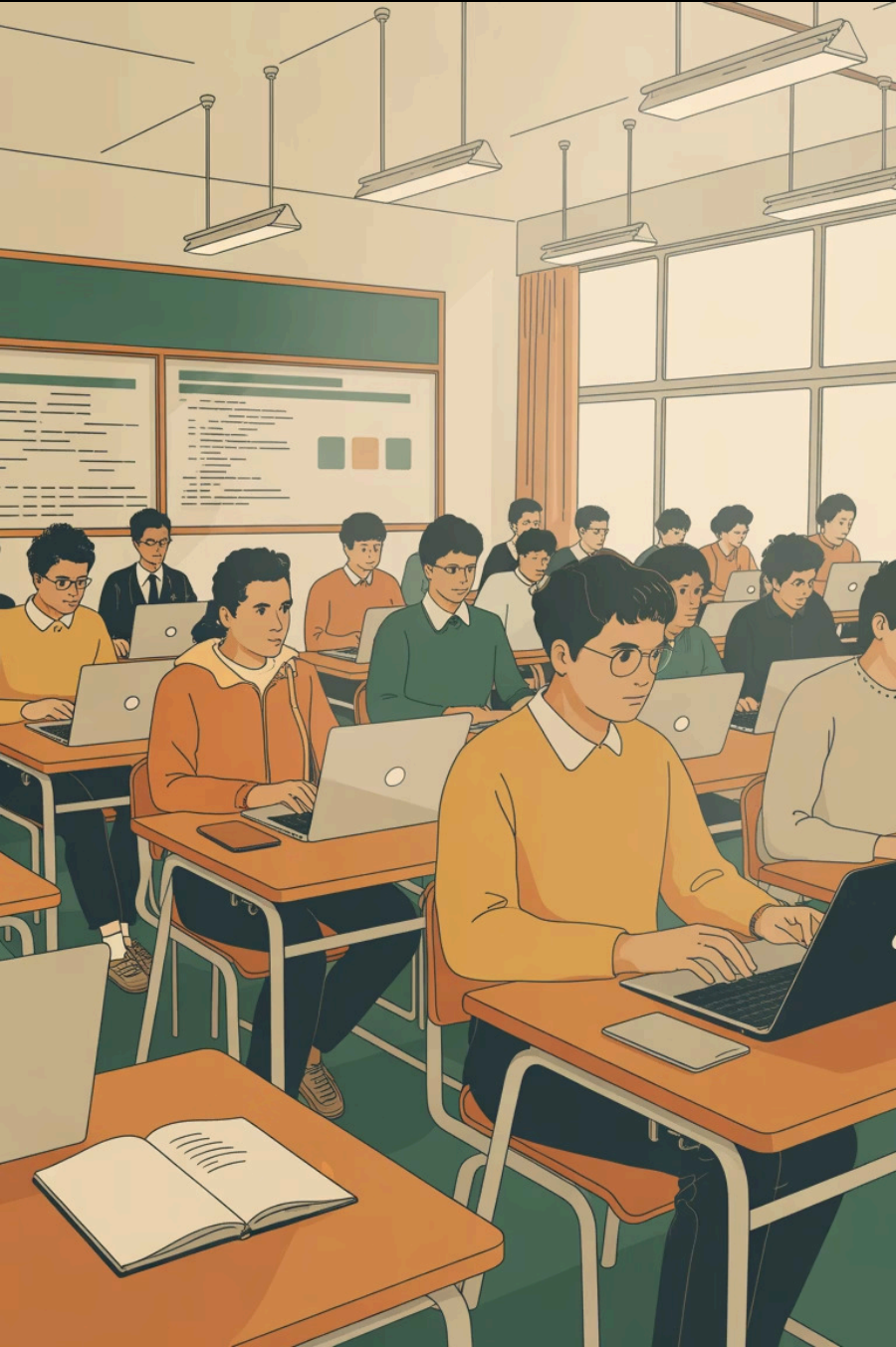




ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Frameworks Backend

Semana 6 — Aula Prática

Tema: Criação de uma API simples utilizando framework back-end.

Carga horária: 3 horas

Prof. Ysmael Marinho Santos Júnior

O que vamos aprender hoje

Objetivos da Aula



Criar uma API simples

Construir uma API REST funcional do zero, com servidor rodando localmente



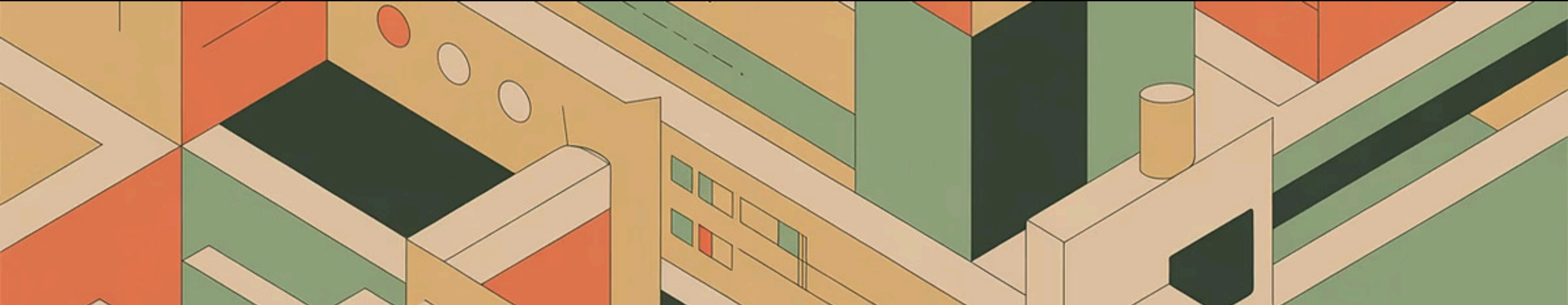
Estrutura em camadas

Compreender Controller, Service e Repository na prática



Endpoints GET e POST

Implementar, testar e validar rotas reais com dados JSON



O que é uma API REST?

Conceito Simples

Uma API é um **contrato de comunicação** entre sistemas. REST é um estilo arquitetural que usa HTTP para transferir dados de forma padronizada.

Cliente → Servidor

O cliente faz uma **requisição HTTP** (GET, POST, PUT, DELETE) e o servidor responde com dados — geralmente em formato **JSON**.

Por que JSON?

Formato leve, legível e universalmente suportado. Facilita a **integração entre sistemas** diferentes — web, mobile, serviços externos.

Cenário e Estrutura do Projeto

O Problema

Sistema de Usuários

- Listar todos os usuários cadastrados
- Criar um novo usuário no sistema
- Retornar dados em formato JSON

Simples, direto e perfeito para aprender os fundamentos!

Estrutura em Camadas

1

Controller

Recebe a requisição HTTP e retorna a resposta

2

Service

Contém a lógica de negócio da aplicação

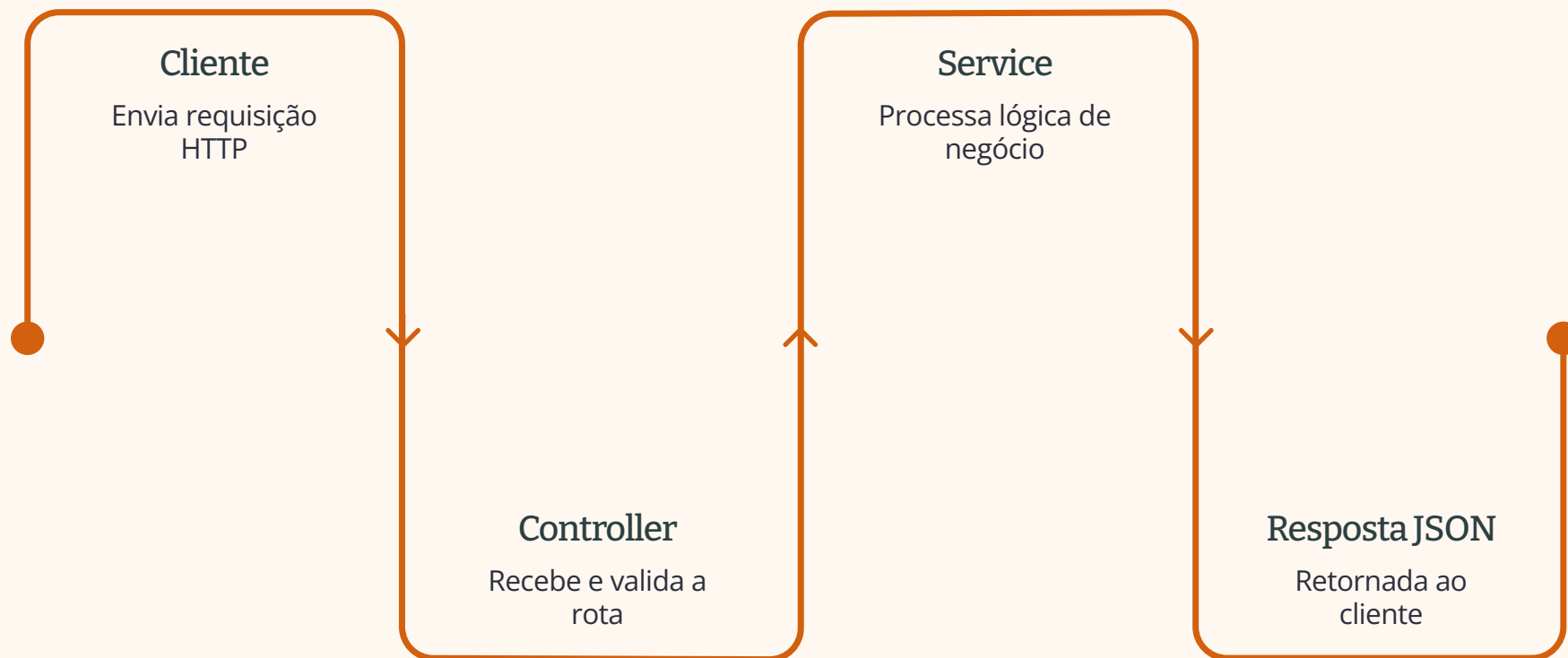
3

Repository (*opcional*)

Abstração do acesso a dados e persistência

Fluxo da Requisição

Entenda como os dados percorrem as camadas da sua API a cada chamada



Cada camada tem uma **responsabilidade única e bem definida**. Essa separação torna o código mais fácil de testar, manter e evoluir ao longo do tempo.

Criando o Projeto e os Endpoints

Setup Inicial

01

Inicializar o projeto

Configure o gerenciador de pacotes e instale o framework escolhido

02

Criar o servidor

Defina a porta e suba o servidor local — teste no navegador

03

Organizar as pastas

Estruture `controllers/`, `services/` e `routes/`

Endpoints Criados

GET /users

Retorna a lista de todos os usuários cadastrados em formato JSON

POST /users

Recebe dados no corpo da requisição e cria um novo usuário

Exemplo de Requisição JSON

Enviando dados — POST /users

```
{  
  "name": "Maria Silva",  
  "email": "maria@email.com"  
}
```

O corpo da requisição (**body**) é enviado no formato JSON com os campos obrigatórios do usuário.

Resposta do servidor — 201 Created

```
{  
  "id": 1,  
  "name": "Maria Silva",  
  "email": "maria@email.com",  
  "createdAt": "2024-01-15"  
}
```

O servidor retorna o recurso criado com status HTTP **201** e os dados persistidos.

  **Dica:** Campos obrigatórios como `name` e `email` devem sempre ser validados antes de salvar. Nunca confie cegamente nos dados do cliente!

Testando a API

Ferramentas de Teste

Postman

Interface gráfica intuitiva para criar, salvar e organizar coleções de requisições HTTP

Insomnia

Alternativa leve e moderna ao Postman, com suporte a ambientes e variáveis

Checklist de Testes

- **GET /users** → retorna lista vazia ou com dados
- **POST /users** → envia JSON e verifica status 201
- **GET /users** novamente → verifica se o usuário aparece
- Teste com campos faltando → comportamento esperado?
- Verifique os **status codes** retornados

Boas Práticas e Próximos Passos

✓ Boas Práticas Iniciais

→ Separação de camadas

Cada camada deve ter uma única responsabilidade — nunca misture lógica de negócio no Controller

→ Organização de código

Nomes descritivos, pastas coerentes e arquivos pequenos facilitam a manutenção

→ Status HTTP corretos

Use 200, 201, 400, 404 e 500 de forma adequada

🚀 Possíveis Evoluções

Banco de dados

Persistir dados reais com PostgreSQL, MySQL ou MongoDB

Validação

Validar campos obrigatórios, tipos e formatos antes de processar

Middleware

Autenticação, logging e tratamento de erros centralizado

Mão na massa!

Atividade Prática & Encerramento

Desafio da Aula

01

Crie sua própria API

Escolha um tema (produtos, tarefas, livros...) e estruture as camadas

02

Implemente os endpoints

GET para listar e POST para criar — com resposta JSON

03

Teste no Postman/Insomnia

Valide todos os endpoints e verifique os status codes

04

Suba no GitHub

Crie um repositório, adicione um README e compartilhe o link

O que aprendemos hoje

API REST

Conceito, comunicação cliente-servidor e JSON

Camadas

Controller → Service → Repository

Endpoints

GET e POST funcionando e testados

BOM DESENVOLVIMENTO A TODOS! 🚀